# R3.09 - Cryptographie

Léopold Trémant

BUT Informatique, IUT de Lens, Université d'Artois

**Licence CC BY-NC-SA 4.0**
Ce document peut être librement copié, distribué et transmis, à condition de mentionner l'auteur et de ne pas l'utiliser à des fins commerciales. Il est également possible de le modifier à condition de le partager sous la même licence.

La licence est disponible en ligne : `https://creativecommons.org/licenses/by-nc-sa/4.0/deed.fr`

**Colophon**
Ce document a été rédigé avec l'aide de KOMA-Script et LaTeX en utilisant la classe kaobook.

**Version**
Version 0.1, dernière mise à jour le 9 septembre 2026.

Ultimately, saying that you don't care about privacy because you have nothing to hide is no different from saying you don't care about freedom of speech because you have nothing to say.

– Edward Snowden

# Préface

La ressource « R3.09 – Cryptographie » a pour objectif de fournir aux étudiants les connaissances et compétences nécessaires pour comprendre les principes fondamentaux de la cryptographie, ainsi que leur application dans le domaine de la sécurité informatique.

**Remerciements.** Merci infiniment à Pierre Catoire pour la rédaction complète de la première version de ce cours. Merci également aux étudiant·es du BUT d'Informatique de l'IUT de Lens de toujours accueillir chaleureusement ce cours, malgré les efforts qu'il leur demande.

## Programme

Afin de correspondre aux attentes des étudiant·es, de l'enseignant et du programme national, les compétences visées par ce cours sont plus spécifiquement regroupées en trois catégories : la culture cryptographique, les mathématiques et la programmation. Concrètement, à l'issue de ce cours, les étudiant·es devront être capables de :

- Décrire et alerter sur des failles de sécurité basiques dans un système informatique ;
- Manipuler certains outils mathématiques de théorie des nombres ;
- Sélectionner un algorithme de chiffrement ou de hachage adapté à un contexte donné.

Plus de détails sur le programme et le déroulement de ce cours sont disponibles en Annexe A.1.

## Comment utiliser ce polycopié ?

Le poly est principalement pensé comme version « propre » du TD, avec des explications et des exercices supplémentaires. Les éléments importants sont mis en évidence dans des encadrés ou sont indiqués par des symboles ♥. Les exercices sont regroupés à la fin de chaque chapitre, et les solutions sont disponibles en fin de document.

**Lien avec les TPs.** Bien que les séances de cours alternent entre les mathématiques en TD et la programmation en TP, il est possible que les exercices et études de TP n'y soient pas consignés. [#1] Pour les aspects de programmation, il faut donc aussi se référer aux *notebooks* sur Moodle.

#1 : La Section 1.1 correspond au TP1, mais je n'aurai probablement pas le temps de faire cette transcription pour tous les TPs.

**Contenu supplémentaire ♠** Certaines parties du poly sont marquées par le symbole ♠. Il s'agit de contenu supplémentaire, qui n'est pas nécessaire pour ce cours, mais qui peut être intéressant, notamment d'un point de vue mathématique. Pour rendre leur aspect facultatif plus clair, ces parties sont également indentées et écrites en plus petit.

**Annotations.** La marge contient de nombreuses annotations qui fournissent des précisions ou des digressions. Lorsque ces remarques dépassent le cadre du cours, elles sont marquées par le symbole ♠.

## Limitations de ce polycopié

Attention, il ne faut pas penser qu'après avoir suivi ce cours, vous serez capable d'implémenter vous-même un système de cryptographie sécurisé. Ce cours permet seulement d'acquérir une culture générale sur le sujet, tandis que l'implémentation de systèmes cryptographiques est un domaine d'expertise à part entière. Si on essaie de faire les choses soi-même, il est impossible de ne pas commettre des erreurs qui compromettent la sécurité du système.

**Obsolescence.** Il ne s'agit pas ici d'être à la pointe des dernières avancées en cryptographie, mais de comprendre les principes fondamentaux. La cryptographie est un domaine en constante évolution, [#2] donc je me contente ici de présenter certains concepts de base, qui sont plus durables que les algorithmes eux-mêmes. Il s'agit aussi d'une excuse pour étudier des mathématiques très répandues, comme les structures algébriques et l'arithmétique.

**Collaborer.** Ce document reste un *travail en cours* qui sera complété au fil de l'année. Il sera complété et corrigé en fonction des retours des étudiant·es. Il ne faut surtout pas hésiter à me signaler des erreurs, des coquilles ou des points qui ne sont pas clairs.

#2 : Par exemple, la France a adopté en 2011 une courbe elliptique « de référence ». D'après le site de référence SafeCurves, ces paramètres sont désormais considérés comme peu sûrs, et il est recommandé de ne plus les utiliser. L'ANSSI met régulièrement à jour son Guide de sélection d'algorithmes cryptographiques.

# Table des matières

**Table des matières** **vii**

**1. Introduction à la cryptographie** **1**
1.1. Chiffrer pour assurer la confidentialité . . . 1
1.1.1. Exemple : le chiffre de César . . . 2
1.1.2. Chiffrement par masque jetable . . . 3
1.1.3. Le risque de réutiliser un masque . . . 4
1.1.4. ♥ Conclusion . . . 5
1.2. Définir un secret partagé . . . 6
1.2.1. Échange de clés Diffie-Hellman. . . . 6
1.2.2. ♠ La pratique au-delà des maths . . . 7
1.3. ♠ La cryptographie n'est pas... . . . 7
1.3.1. ... la stéganographie . . . 8
1.3.2. ... la cybersécurité . . . 8
1.4. Exercices du chapitre . . . 8

**2. Arithmétique modulaire** **11**
2.1. Anneau résiduel . . . 11
2.1.1. ♠ Structures algébriques . . . 11
2.1.2. Division euclidienne . . . 12
2.1.3. Nombres modulaires . . . 13
2.1.4. ♠ Une autre définition de l'anneau résiduel . . . 15
2.2. Exercices du chapitre . . . 16

**Appendices** **17**

**A. Informations vis-à-vis du cours** **19**
A.1. Programme du cours . . . 19
A.1.1. Programme national . . . 19
A.1.2. Contrat pédagogique . . . 19
A.2. Ressources complémentaires . . . 21

**Solutions aux exercices** **23**

# 1. Introduction à la cryptographie

Lorsqu'une *source* souhaite envoyer un message à un *destinataire* via un *canal* non sécurisé, comme des ondes (wifi, Bluetooth...) ou un réseau (LAN, téléphone, internet), une *adversaire* peut potentiellement intercepter l'information en transit via le canal. On parle d'attaque *passive* si l'adversaire ne peut pas modifier le message, ou d'attaque *active* sinon. [#1]

#1 : Certaines sources utilisent plutôt les termes *écoute* et *attaque* pour distinguer les attaques passive et active respectivement.

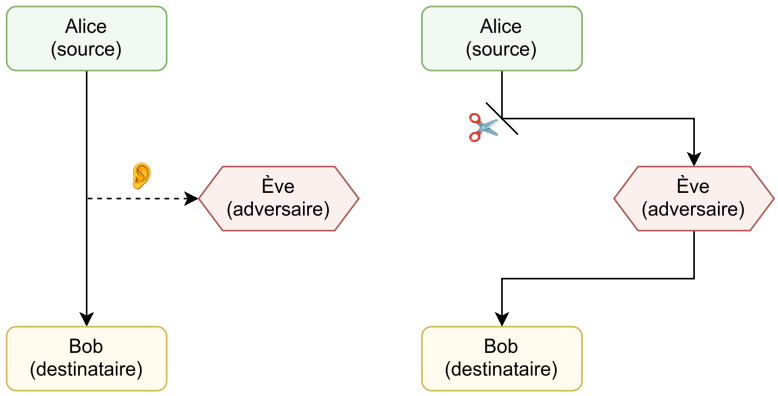



**Figure 1.1. :** Attaque passive (gauche) et attaque active (droite).

Ici, on supposera que l'adversaire est active, c'est-à-dire qu'elle peut intercepter tous les messages, les modifier, les supprimer, les réémettre, etc. Généralement, en cryptographie, la source s'appelle *Alice*, le destinataire *Bob* et l'adversaire *Eve*.

> ♥ En cryptographie, la sécurité de la communication repose sur deux critères : [♠2]
>
> 1. *Confidentialité :* Le message est secret (ou illisible) pour tout agent tiers.
> 2. *Intégrité :* Le message n'a pas été modifié par un tiers.
>
> Le premier empêche les attaques passives, et le second empêche les attaques actives.

♠2 : Dans un contexte plus général de sécurité de l'information, on aurait des critères supplémentaires (voir e.g. l'hexade de Parker).

En Section 1.1, on se concentre sur la confidentialité, en introduisant des méthodes de chiffrement pour rendre les messages transmis inintelligibles. Ces méthodes sont dites *symétriques* car elles reposent sur un secret partagé entre Alice et Bob. En Section 1.2, on présente la procédure de Diffie-Hellman, qui permet justement de définir un secret partagé sur un canal ouvert.

## 1.1. Chiffrer pour assurer la confidentialité



Pour éviter les écoutes, Alice et Bob utilisent un *algorithme de chiffrement*.

a. Iels définissent ensemble une *clé de chiffrement* 🔑.
b. Alice *chiffre* 🔒 le message en clair (« Bonjour ») grâce à la clé 🔑.
c. Alice envoie le message chiffré (`0x01ab`) à Bob. [#3]
d. Bob *déchiffre* 🔓 le message en utilisant la clé 🔑.

#3 : Dans ce document, on tente de distinguer les messages en clair des messages chiffrés en utilisant la police sans-serif pour les messages en clair et la `police monospace` pour les messages chiffrés. On s'efforce aussi de baliser ces types de messages différemment.

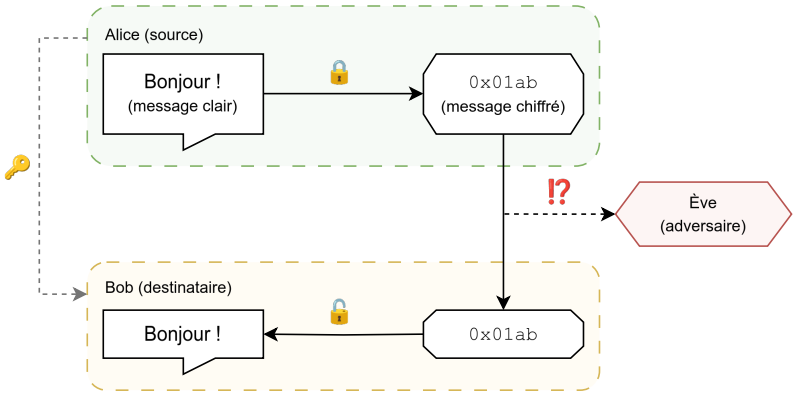



**Figure 1.2. :** Envoi d'un message chiffré & déchiffré avec une clé de chiffrement échangée secrètement.

Le message chiffré (`0x01ab`) est complètement incompréhensible. Même si Eve intercepte le message, elle ne pourra pas le comprendre sans le déchiffrer 🔓, ce qui est impossible sans connaître la clé 🔑.

> ⚠ La confidentialité ne doit pas dépendre du secret de l'algorithme de chiffrement, seulement du secret de la clé. Cela fait partie des *principes de Kerckhoffs*. [#4]

#4 : *Principes de Kerchoffs* :

- l'algorithme repose sur le secret de la clef et non sur le secret de l'algorithme ;
- le déchiffrement sans clef ne doit pas être une option raisonnable ;
- étant donné un texte en clair et un texte chiffré il est impossible de retrouver la clef.

Mathématiquement, on appelle la clé $k$ 🔑. Elle définit une fonction de chiffrement $E_k$ (comme *encryptor* 🔒) et une fonction de déchiffrement $D_k$ (comme *decryptor* 🔓). Le processus ci-dessus devient (Figure 1.3) :

a. Iels définissent ensemble une *clé de chiffrement* $k$.

b. Alice *chiffre* son message $m$ via la fonction $E_k$ pour obtenir le message chiffré $c := E_k(m)$.

c. Alice envoie le message chiffré $c$ à Bob.

d. Bob *déchiffre* le message chiffré $c$ via la fonction $D_k$ pour retrouver le message original $d := D_k(c) = D_k \circ E_k(m) = m$.

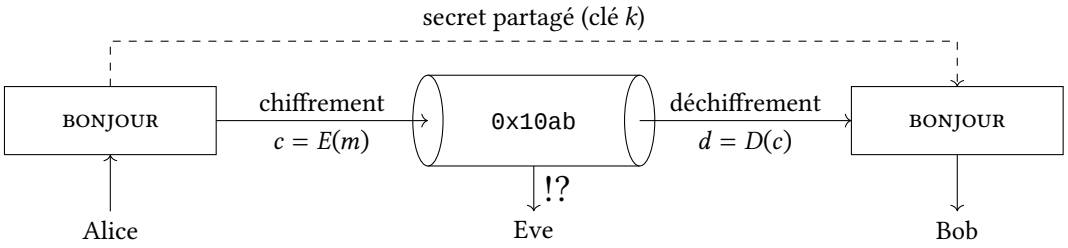



**Figure 1.3. :** Reproduction mathématisé du processus de chiffrement et déchiffrement de la Figure 1.2.

### 1.1.1. Exemple : le chiffre de César

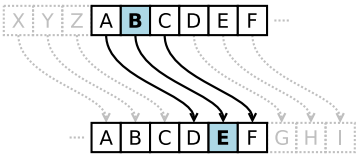



**Figure 1.4. :** Illustration du chiffre de César avec la clé `D` (ou 3). Source *Wikimedia Commons*.

Un des plus anciens algorithmes de chiffrements est le chiffre de César. Il consiste à décaler chaque lettre d'un certain nombre de positions dans l'alphabet. La clé est donc simplement le décalage, donné par le chiffrement de la lettre « A ».

Par exemple, avec une clé `D` (i.e. un décalage de 3), « A » devient `D`, « B » devient `E`, etc (Figure 1.4). Le fonctionnement est circulaire, de sorte que « X » devient `A`, « Y » devient `B` et « Z » devient `C`. Ainsi, on a le chiffrement

« Bonjour » $\xrightarrow{\text{décalage de 3}}$ `Erqmrxu`

Étant donné que la clé de chiffrement permet aussi le déchiffrement, on parle de *chiffrement symétrique.* ♠5

♠5 : D'autres exemples de chiffrements symétriques sont :
- le chiffrement par *permutation* ;
- le chiffrement de Vigenère ;
- le chiffrement de Vernam ;
- le chiffrement de Hill ;
- le chiffrement *Enigma* utilisé par la marine Allemande durant la seconde guerre mondiale.
- le *Digital Encryption Standard* appelé DES ;
- le *Advanced Encryption Standard* appelé AES.

*Exemple* 1.1. Le texte

`Iutzk-suo, Sayk, r'nussk gad sorrk zuaxy`

peut être déchiffré en

« Conte-moi, Muse, l'homme aux mille tours »

avec la clé `G` (ou 6) du chiffre de César.

Aujourd'hui, ce chiffrement est très facile à casser, même avec des outils gratuits en ligne comme dCode. Ces décryptages mêlent généralement une analyse statistique et un *brute-force* de la clé. Comme il n'y a que 26 clés possibles, même un humain peut facilement tester toutes les clés et retrouver le message original.

**Exercice 1.1**

Dans l'outil en ligne dCode :

1. Déchiffrer automatiquement le message `Erqmrxu`. Retrouve-t-on le message original « Bonjour » ?
2. Chiffrer « Zut » avec un décalage de 10.
   a) Quel est le message chiffré obtenu ?
   b) Déchiffrer automatiquement le message obtenu. Retrouve-t-on le message original ?
   c) Qu'en conclure sur la sécurité du chiffre de César ?

## 1.1.2. Chiffrement par masque jetable

Pour éviter que ce chiffrement soit trop facile à casser, on peut utiliser un *masque jetable* (ou *one-time pad* en anglais). Il s'agit d'une clé de chiffrement aléatoire, de la même longueur que le message à chiffrer.

Comme remarqué en Exercice 5, utiliser une clé d'une longueur proche de celle du message rend le décryptage #6 plus difficile. En utilisant une clé purement aléatoire et de la même longueur que le message, on obtient un chiffrement théoriquement incassable.

#6 : Il faut distinguer déchiffrement et décryptage : le premier suppose de connaître la clé et consiste simplement en l'application de l'algorithme de déchiffrement ; le second suppose de trouver la clé avant de déchiffrer.

**Images 1-bit.** Pour des données informatiques génériques, l'alphabet n'est pas composé de 26 lettres, mais de bits ! On va donc considérer des images avec 1 bit par pixel, en noir et blanc (de valeurs 0 et 1 respectivement). On voit une telle image en Figure 1.5. Le visage est blanc (valeur 1), tandis que le fond, les yeux et la bouche sont noirs (valeur 0).



**Figure 1.5. :** Exemple d'image 1-bit en clair.

**Le xor** est un opérateur binaire associatif et commutatif (comme and et or) défini par la table de vérité ci-contre (Table 1.1). Autrement dit, pour un bit $b$,

- $b \oplus 0 = b$ préserve la valeur du bit ;
- $b \oplus 1 = \neg b$ inverse la valeur du bit.

En Python, cet opérateur est implémenté sur les entiers (`int`) avec le symbole `^`.

| $\oplus$ | **0** | **1** |
|---|---|---|
| **0** | 0 | 1 |
| **1** | 1 | 0 |

**Table 1.1. :** Table de vérité de l'opérateur xor.



image claire $m$ | clé $k$ | image chiffrée $c = m \oplus k$

**Figure 1.6. :** Chiffrement xor.

**Chiffrement xor.** En Figure 1.6, on voit le résultat du chiffrement du smiley (image 1-bit) par un masque jetable, la clé. ♠7 La clé est composée de pixels aléatoires (où être blanc ou noir est équiprobable), mis à part deux bandes horizontales.

♠7 : Ce chiffrement xor est aussi appelé chiffrement de Vernam.

- La bande blanche (valeurs 1) a inversé les pixels du smiley : le noir est devenu blanc, et réciproquement.
- La bande noire (valeurs 0) n'a pas modifié les pixels.
- Dans les bandes aléatoires, on ne distingue plus du tout l'image originale.

C'est bien le comportement attendu étant donné les identités `` `b^1 = !b` `` et `` `b^0 = b` ``. La partie aléatoire est forcément impossible à décrypter, #8 puisque cela reviendrait à deviner quels pixels ont été inversés, et lesquels n'ont pas été modifiés, or ces pixels ont été choisis aléatoirement. Dans ces bandes aléatoires, on ne peut pas retrouver l'image originale sans connaître la clé (i.e. le masque).

#8 : Le décryptage consiste à *retrouver la clé*, puis à déchiffrer le message.

**Déchiffrement xor.** Pour déchiffrer l'image, il suffit d'appliquer le masque une seconde fois :

```
`d = c \xor k`
```

En effet, un calcul direct #9 vérifie $d = m$ avec

$$d = c \oplus k = (m \oplus k) \oplus k = m \oplus (k \oplus k) = m \oplus 0 = m, \tag{1.1}$$

où on utilise la diagonale de la table de vérité, $0 \oplus 0 = 1 \oplus 1 = 0$. On voit le résultat en Figure 1.7.

#9 : Le xor n'est défini que pour des bits, mais on l'étend à des vecteurs de bits en l'appliquant bit par bit.



image chiffrée $c = m \oplus k$ | clé $k$ | image déchiffrée $d = c \oplus k$

**Figure 1.7. :** Déchiffrement xor.

### 1.1.3. Le risque de réutiliser un masque

**Il ne faut jamais réutiliser un masque jetable !**

Si un masque xor est réutilisé, on peut facilement retrouver (une partie au moins) du message original.

Lorsqu'on calcule le XOR des deux messages chiffrés, on obtient le XOR des deux messages originaux. En effet, si deux messages $m_1$ et $m_2$ sont chiffrés avec le même masque $k$, alors les messages chiffrés sont $c_1 = m_1 \oplus k$ et $c_2 = m_2 \oplus k$. Dans ce cas, on a

$$\begin{aligned} c_1 \oplus c_2 &= (m_1 \oplus k) \oplus (m_2 \oplus k) \\ &= m_1 \oplus m_2 \oplus (k \oplus k) \\ &= m_1 \oplus m_2 \oplus 0 \\ &= m_1 \oplus m_2. \end{aligned}$$

par commutativité de $\oplus$

d'après la table de vérité

On voit cela en Figure 1.8, où deux images différentes ont été chiffrées avec le même masque. Dans les zones où le smiley comporte de grands aplats, on voit apparaître une forme de bateau guerrier.

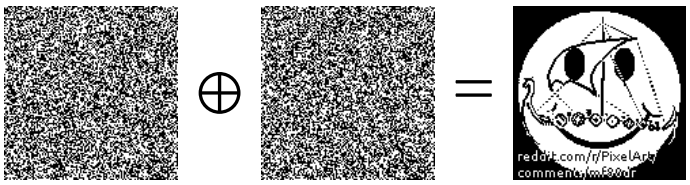



image chiffrée 1 $c_1 = m_1 \oplus k$ | image chiffrée 2 $c_2 = m_2 \oplus k$ | XOR des images $c_1 \oplus c_2 = m_1 \oplus m_2$

**Figure 1.8. :** Récupération d'information à partir d'images chiffrées avec le même masque XOR.

Bien que l'on ne soit pas en mesure de récupérer l'intégralité de l'image de bateau si l'on ne connait pas parfaitement l'image de smiley, on réussit à récupérer une partie de l'information originale (à savoir : que c'est une image de bateau), ce qui est déjà un problème de sécurité.

### 1.1.4. ♥ Conclusion

On a donc ainsi décrit deux méthodes de chiffrement, le chiffre de César et le chiffrement XOR par masque jetable.



**Figure 1.9. :** Un disque de chiffrement César (source *Wikimédia Commons*).

#### Chiffre de César

- clé : nombre de décalage (1 à 25) ;
- chiffrement : décalage circulaire des lettres de l'alphabet ;
- déchiffrement : décalage circulaire inverse ;
- sécurité : faible, la clé est trop courte.

#### Masque XOR

- clé : masque aléatoire de la même longueur que le message ;
- chiffrement : XOR du message avec le masque ;
- déchiffrement : XOR du message chiffré avec le masque ;
- sécurité : le masque ne doit jamais être réutilisé.

Le chiffre de César n'est adapté qu'au texte et ne sert comme illustration pédagogique (ou pour des jeux d'énigmes). Le chiffrement XOR est plus général et peut s'appliquer à n'importe quel type de données, mais il est difficile à mettre en œuvre dans la pratique, car il faut générer un masque aléatoire de la même longueur que le message. [#10] Il faut également trouver un moyen de partager ce masque sans qu'il ne puisse être intercepté.

#10 : En pratique on utilisera donc une méthode *pseudo-aléatoire*, c'est-à-dire un générateur de nombres qui semblent aléatoires, mais qui sont en fait déterministes et donc reproductibles. Ce générateur prend un paramètre qui s'appelle la *graine* et qui sera le secret partagé entre Alice et Bob.

## 1.2. Définir un secret partagé



Dans la section précédente (et en Figure 1.3), la clé de chiffrement $k$ est un secret partagé entre Alice et Bob. Pour assurer sa confidentialité, on doit donc utiliser une procédure d'*échange de clés*, qui permet de définir un secret partagé sans qu'il ne soit jamais directement transmis sur le canal de communication.

### 1.2.1. Échange de clés Diffie-Hellman.

Dans le document de référence `RFC 2631`, ♠11 on lit

> `the shared secret ZZ is generated as follows:`
>
> `ZZ = g ^ (xb * xa) mod p`
>
> `Note that the individual parties actually perform the computations:`
>
> `ZZ = (yb ^ xa) mod p = (ya ^ xb) mod p`
>
> `where ^ denotes exponentiation`

♠11 : Ce document date de 1999 et est en partie obsolète. L'idée générale de cet échange de clés demeure inchangée, mais on préfère maintenant des courbes elliptiques à l'exponentiation modulaire.

L'opération `mod` est le modulo, c'est-à-dire le reste de la division euclidienne, que nous définirons rigoureusement plus tard et que nous allons noter avec un symbole %. Par exemple, $7\%3 = 1$ car $7 = 2 \times 3 + 1$.

Autrement dit, en utilisant des paramètres publics de modulus $p$ et de générateur $G$ pré-définis, Alice et Bob peuvent définir un secret partagé $Z$ en suivant la procédure suivante :

| *Alice* | *Bob* |
|---|---|
| ▸ choisit un secret $a$ (`xa`) | ▸ choisit un secret $b$ (`xb`) |
| ▸ calcule $A = G^a\%p$ (`ya`) | ▸ calcule $B = G^b\%p$ (`yb`) |
| ▸ envoie $A$ à Bob | ▸ envoie $B$ à Alice |
| ▸ calcule $Z_a = B^a\%p$ (`ZZ`) | ▸ calcule $Z_b = A^b\%p$ (`ZZ`) |

Le secret partagé est $Z = Z_a = Z_b = G^{a\times b}\%p$.

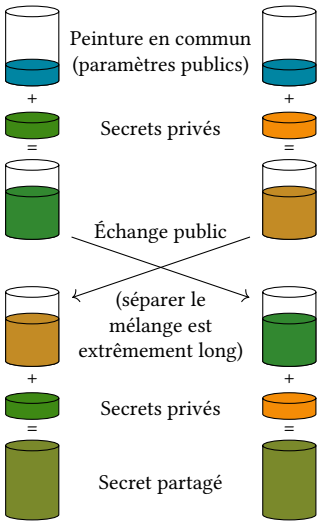



**Figure 1.10. :** Illustration de l'échange de clés Diffie-Hellman avec des couleurs (inspiré d'une source *Wikimedia Commons*).

Les secrets privés $a$ et $b$ ne sont pas échangés, mais Alice et Bob peuvent tout de même calculer le même secret partagé $Z$ à partir des nombres publics $A$ et $B$. Pour s'assurer que la procédure est sécurisée, il faut que le calcul de $Z$ à partir de $A$ et $B$ soit difficile pour un tiers. C'est ce que nous allons étudier dans la suite du cours.

Une analogie avec les couleurs est souvent utilisée pour expliquer cette procédure. Les paramètres publics sont représentés par une couleur $G$ choisie publiquement, et les nombres secrets privés sont des teintes de couleur $a$ et $b$. Après avoir mélangé la base publique avec la clé/couleur privée, Alice et Bob obtiennent chacun de leur côté un mélange, respectivement $A$ et $B$. Après avoir échangé ces mélanges, ils peuvent chacun de leur côté re-mélanger le mélange reçu avec leur clé/couleur privée pour obtenir le même mélange final $Z_a = Z_b = Z$, qui est le secret partagé. Même si Ève connaît les couleurs échangées $A$ et $B$, en ôter la couleur $G$ qu'elles contiennent pour retrouver $a$ ou $b$ est pratiquement impossible. Elle ne peut donc pas constituer la couleur finale.

Effectuons un calcul de clé partagée avec des nombres relativement petits, pour vérifier que la procédure fonctionne.

**Exercice 1.2**

Calculer $A$, $B$, $Z_a$ et $Z_b$ avec les paramètres suivants :

$$G = 5, \qquad p = 23, \qquad a = 2, \qquad b = 3.$$

En binôme, répartissez-vous les rôles d'Alice et de Bob, et vérifiez que vous obtenez bien le même secret partagé $Z$.

L'objectif du cours va être de répondre aux questions suivantes.

1. Est-ce que Alice et Bob obtiennent bien le même secret partagé $Z = Z_a = Z_b$ ? [#12]
2. Comment choisir $p$ et $G$?
3. Pourquoi est-il difficile de retrouver $Z$ à partir de $A$ et $B$?
   - Quelles sont les valeurs possibles pour $A$ et $B$?
   - Y a-t-il des contraintes sur $a$ et $b$?
4. Est-ce que toutes ces étapes sont rapides ? [#13]

#12 : Autrement dit, a-t-on $(G^a \% p)^b \% p = (G^b \% p)^a \% p$?

#13 : On devra définir rigoureusement la notion de rapidité !

Les outils mathématiques que nous allons développer permettront ensuite de comprendre la procédure RSA, qui assure la protection contre les attaques actives.

**Le TD1 continue en Section 2.1.**

### 1.2.2. ♠ La pratique au-delà des maths

En pratique, la procédure Diffie-Hellman n'est qu'une des « sources d'entropie ». Elle est combinée avec des informations publiques (dont un nombre aléatoire) pour obtenir une suite d'octets qui est fournie à une fonction de hachage. Dans la spec, on lit

```
`KM = H ( ZZ || OtherInfo)`
```

## 1.3. ♠ La cryptographie n'est pas...

Les procédures cryptographiques permettent d'avoir de la *confidentialité*, des *authentifications*, *signatures électroniques*...

Sans surprise, pouvoir envoyer des messages sans être compris par d'autres est d'une importance capitale. Depuis l'antiquité, les hommes souhaitent envoyés des messages secrets pour des raisons commerciales, militaires... Aujourd'hui, la cryptographie est omniprésente dans nos vies, qu'il s'agisse de transactions bancaires, d'échanges avec des sites web, d'identifiants personnels... Pour certains produits, il s'agit d'ailleurs d'un argument de vente : la plupart des applications de messagerie instantanée se présentent comme chiffrées de bout en bout, sous-entendant que la confidentialité est assurée (par exemple WhatsApp).

### 1.3.1. ... la stéganographie

La stéganographie consiste à dissimuler une information sensible dans un média d'apparence anodine. De cette sorte, le message n'éveille pas de soupçons même s'il est intercepté. Dans l'antiquité, par exemple, en tatouant le crâne du messager et en attendant la repousse des cheveux. Plus récemment, en écrivant à l'encre invisible ou en utilisant des tiroirs à double fond. À l'ère du numérique, c'est une méthode beaucoup utilisée pour les énigmes, comme dans les jeux en réalité alternée (ou ARG), avec des images cachées dans les bits de poids faible d'une autre image, ou avec des messages camouflés dans des spectrogrammes.

En cryptographie, le premier principe de Kerckhoffs requiert que l'algorithme de chiffrement (i.e. le procédé de rendre secrètes les données) peut être connu sans que le secret soit facilement brisé. En stéganographie, la dissimulation est basée sur le secret de la méthode.

Néanmoins, la sténographie reste utilisée de nos jours. Dans les pays où l'envoi de messages chiffrés est surveillé ou interdit, il est plus pertinent de ne pas éveiller les soupçons. Même dans les pays dits libéraux, la cryptographie a longtemps été réservée à l'usage militaire (jusque dans les années 80 en France). Aujourd'hui, son autorisation semble précaire, notamment avec projet de loi controversé Chat Control, il est donc difficile de penser que la sténographie cessera un jour d'être pertinente.

### 1.3.2. ... la cybersécurité

La cybersécurité est un ensemble de pratiques et de techniques pour sécuriser les systèmes d'informations, alors que la cryptographie se concentre sur le brouillage des données. En particulier, une grosse part des failles de cybersécurité sont humaines, et ne sont pas considérées en cryptographie.

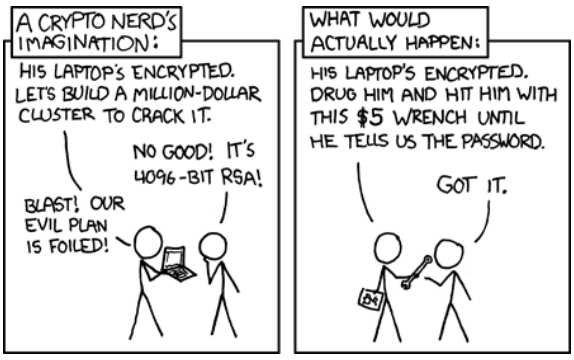



**Figure 1.11. :** XKCD 538

La sécurité est une chaîne, et il suffit d'un maillon faible pour la briser. En général, la cryptographie n'est pas ce maillon faible.

## 1.4. Exercices du chapitre

**Exercice 1.1**

Dans l'outil en ligne dCode :

1. Déchiffrer automatiquement le message `` `Erqmrxu` ``. Retrouve-t-on le message original « Bonjour » ?
2. Chiffrer « Zut » avec un décalage de 10.
   a) Quel est le message chiffré obtenu ?
   b) Déchiffrer automatiquement le message obtenu. Retrouve-t-on le message original ?
   c) Qu'en conclure sur la sécurité du chiffre de César ?

**Exercice 1.2**

Calculer $A$, $B$, $Z_a$ et $Z_b$ avec les paramètres suivants :

$$G = 5, \qquad p = 23, \qquad a = 2, \qquad b = 3.$$

En binôme, répartissez-vous les rôles d'Alice et de Bob, et vérifiez que vous obtenez bien le même secret partagé $Z$.

# Arithmétique modulaire | 2.

## 2.1. Anneau résiduel

Les entiers relatifs forment un anneau : on peut les additionner, soustraire et multiplier entre eux. Il y a un 0 et un 1, les éléments neutres respectivement pour l'addition/soustraction et pour la multiplication. C'est-à-dire qu'ils vérifient

$$\forall x \in \mathbb{Z}, \quad x + 0 = 0 + x = x \quad \text{et} \quad x \times 1 = 1 \times x = x.$$

♠ Pour former un anneau, il faut aussi que l'addition soit commutative, et il faut avoir une relation de distributivité entre + et ×. C'est bien le cas ici.

En revanche, *on ne peut généralement pas diviser* un nombre par un autre. Dans ce cours, on n'utilisera jamais de fraction ou de division.

### 2.1.1. ♠ Structures algébriques

On peut construire différentes structures algébriques, avec plus ou moins d'opérations possibles. Deux d'entre elles nous intéresseront particulièrement : le *groupe* et l'*anneau*.

**Un monoïde** $(E, \star, e)$ est un ensemble muni d'une opération binaire associative $\star$ et d'un élément neutre $e$ pour cette opération. Autrement dit, un monoïde possède les propriétés

- *Associativité :* $\forall x, y, z \in E, \quad (x \star y) \star z = x \star (y \star z)$
- *Élément neutre :* $\forall x \in E, \quad x \star e = e \star x = x$

Par exemple, l'ensemble des nombres naturels $\mathbb{N}$ avec l'addition $+$ et le neutre 0 forme un monoïde. C'est d'ailleurs un monoïde *commutatif*, car l'addition est *commutative* :

$$\forall x, y \in \mathbb{N}, \quad x + y = y + x.$$

Un autre exemple est $(M_n(\mathbb{R}), \times, I_n)$, l'ensemble des matrices carrées de taille $n > 1$ avec $n$ fixé, avec la multiplication matricielle comme opération et la matrice identité $I_n$ comme élément neutre. Dans ce cas, le monoïde n'est pas commutatif : pour deux matrices $A, B \in M_n(\mathbb{R})$, en général $A \times B \neq B \times A$.

**Un groupe** $(G, \star, e)$ est un monoïde dans lequel chaque élément possède un inverse pour l'opération $\star$. Autrement dit, un groupe possède les propriétés d'un monoïde et en plus

- *Inverse :* $\forall x \in G, \exists y \in G, \quad x \star y = y \star x = e$

On note cet inverse $y$ par $x^{-1}$, et on a donc $x \star x^{-1} = x^{-1} \star x = e$.

Par exemple, $(\mathbb{Z}, +, 0)$ est un groupe commutatif. Dans ce cas, l'« inverse » d'un élément $x \in \mathbb{Z}$ est son opposé $-x$ car $x + (-x) = 0$. Il y a aussi, même si on ne n'utilisera pas de nombres à virgule dans ce cours, le groupe $(\mathbb{R}^*, \times, 1)$ des nombres réels non nuls avec la multiplication comme opération et 1 comme élément neutre. Dans ce cas, l'« inverse » d'un élément $x \in \mathbb{R}^*$ est son inverse multiplicatif $1/x$ car $x \times (1/x) = 1$.

En cryptographie, les *courbes elliptiques* sont des groupes très importants. Elles admettent une loi d'addition avec les règles de la tangente et de la sécante (illustrée en Figure 2.1). L'élément neutre est un point fictif à l'infini, et l'inverse d'un point est son symétrique par rapport à l'axe des abscisses.

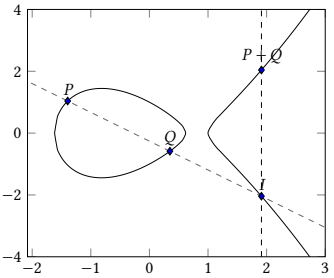



**Figure 2.1. :** La courbe elliptique $Y^2 = X^3 - 2X + 1$, et la règle d'addition de la sécante.

**Un anneau** $(A, +, \times, 0, 1)$ est un ensemble muni de deux opérations binaires $+$ et $\times$ qui vérifient les propriétés suivantes :

- $(A, +, 0)$ est un groupe commutatif
- $(A, \times, 1)$ est un monoïde
- *Distributivité :* $\forall x, y, z \in A$,

$$x \times (y + z) = (x \times y) + (x \times z) \quad \text{et} \quad (y + z) \times x = (y \times x) + (z \times x)$$

Par exemple, $(\mathbb{Z}, +, \times, 0, 1)$ est un anneau. L'ensemble des matrices carrées $M_n(\mathbb{R})$ avec la multiplication matricielle et l'addition matricielle est aussi un anneau, mais pas commutatif : en général $A \times B \neq B \times A$ pour deux matrices $A, B \in M_n(\mathbb{R})$.

**Un corps** $(K, +, \times, 0, 1)$ est un anneau dans lequel tout élément non nul possède un inverse multiplicatif. Autrement dit, un corps possède les propriétés d'un anneau et en plus

- *Inverse multiplicatif :* $\forall x \in K, x \neq 0, \exists y \in K, \quad x \times y = y \times x = 1$

i.e. $(K^*, \times, 1)$ est un groupe multiplicatif, où $K^* = K \setminus \{0\}$.

Par exemple, $(\mathbb{Q}, +, \times, 0, 1)$ est un corps. L'ensemble des nombres réels $\mathbb{R}$ et l'ensemble des nombres complexes $\mathbb{C}$ sont aussi des corps.

Dans ce cours, on verra plus tard un exemple de corps assez surprenant : l'anneau quotient $\mathbb{Z}_p$ pour $p$ premier, qui est un corps fini. Cela indique que l'ensemble des nombres (modulo $p$) non nuls $\mathbb{Z}_p^*$ forme un groupe multiplicatif, où tout nombre (modulaire) [#1] admet un inverse multiplicatif *qui reste un nombre entier.*

#1 : Si cette notion de « nombre modulaire » semble floue, c'est normal ! On définit ça proprement juste après.

## 2.1.2. Division euclidienne

**Définition 2.1.** Étant donné un entier relatif $x \in \mathbb{Z}$ et un entier naturel $n$ non nul $n \in \mathbb{N}^*$, on appelle *division euclidienne* de $x$ par $n$ le fait de déterminer [♠2] deux entiers $q, r$ tels que

$$x = qn + r, \qquad 0 \leq r < n.$$

On appelle $q$ le *quotient* et $r$ le *reste* de cette division.

♠2 : Comme souvent en mathématiques, cette définition demande de prouver qu'elle est bien posée. On peut par exemple procéder par récurrence sur $x$, avec $n$ fixé.

Cette décomposition est unique : pour un $x$ et un $n$ donnés, il n'existe qu'un seul couple $(q, r)$ qui vérifie ces conditions. Dans ce cours on notera

$$q = x \div n \quad \text{et} \quad r = x\%n$$

respectivement le quotient et le reste de la division euclidienne de $x$ par $n$.

♠ On pourrait également choisir $n < 0$, auquel cas le reste $r$ reste positif et vérifie $0 \leq r < |n|$. Techniquement, c'est une différence avec le *modulo*, qui lui devient alors négatif, mais cette différence est futile pour nous puisque nous n'utiliserons jamais de diviseur négatif. En Python, c'est le modulo qui est calculé.

En Python, la division euclidienne peut s'effectuer de deux manières :

```
>>> q = x // n
>>> r = x % n
```

```
>>> q, r = divmod(a, b)
```

**La cryptographie n'utilise pas de fractions !**

Si vous voyez apparaître des fractions ou des nombres à virgule dans vos calculs/codes, il y a forcément une erreur quelque part.

Il ne faut pas confondre cette division euclidienne avec la division rationnelle, qui elle peut donner un résultat à virgule. Par exemple, $7/3 = 2.333...$ mais $7 \div 3 = 2$. C'est pareil en Python, `7 / 3 = 2.333...` mais `7 // 3 = 2`. D'ailleurs, `6 // 3 = 2` est bien un `int`, mais `6 / 3 = 2.0` est un `float`

même si la partie décimale est nulle. On utilisera donc toujours la division euclidienne pour garder des `int`.

*Exemple* 2.2.

| dividende | diviseur | décomposition | quotient | reste |
|---|---|---|---|---|
| 48 | 2 | $48 = 24 \times 2 + 0$ | 24 | 0 |
| 47 | 2 | $47 = 23 \times 2 + 1$ | 23 | 1 |
| 63 | 5 | $63 = 12 \times 5 + 3$ | 12 | 3 |
| 29 | 8 | $29 = 3 \times 8 + 5$ | 3 | 5 |
| 5 | 63 | $5 = 0 \times 63 + 5$ | 0 | 5 |

**Exercice 2.1**

Sachant que l'on a $96\,842 = 256 \times 375 + 842$, déterminer, sans faire la division, le reste de la division du nombre 96 842 par chacun des nombres 256 et 375.

### 2.1.3. Nombres modulaires

Soit $n \in \mathbb{N}^*$. On dit que deux entiers relatifs $x$ et $y$ sont *congrus modulo n* si $x\%n = y\%n$. On note alors

$$x \equiv y \mod n.$$

Cela revient à dire [#3] qu'il existe un entier relatif $k \in \mathbb{Z}$ tel que $x = y + kn$. Formellement, on écrit que pour tous $x, y \in \mathbb{Z}$ et $n \in \mathbb{N}^*$,

$$x \equiv y \mod n \Leftrightarrow \exists k \in \mathbb{Z}, \quad x = y + kn. \tag{2.1}$$

#3 : *Preuve* – On note $x = q \times n + r$ et $y = q' \times n + r'$. Par définition de la congruence, si $x \equiv y \mod n$, on a $r = r'$, donc $x = q \times n + y - q' \times n = y + (q - q') \times n$ et donc $k = q - q'$. Réciproquement, si $x = y + k \times n$, alors $r - r' = k' \times n$ avec $k' = q' - q + k$. Comme $0 \leq r, r' < n$, on a $|r - r'| < n$, i.e. $|k'| \times n < n$. Nécessairement, $k' = 0$, donc $r = r'$ et donc $x\%n = y\%n$. ■

**Proposition 2.3** (Stabilité algébrique)**.** *La congruence est stable pour les opérations algébriques d'addition et de multiplication. C'est-à-dire que si* $x_1 \equiv x_2 \mod n$ *et* $y_1 \equiv y_2 \mod n$*, alors*

$$x_1 + y_1 \equiv x_2 + y_2 \mod n \quad \textit{et} \quad x_1 \times y_1 \equiv x_2 \times y_2 \mod n.$$

En remplaçant $y_1$ par $-y_1$ et $y_2$ par $-y_2$, on obtient aussi la stabilité pour la soustraction,

$$x_1 - y_1 \equiv x_2 - y_2 \mod n.$$

On en déduit aussi que pour toute puissance $k \in \mathbb{N}$,

$$x_1 \equiv x_2 \mod n \quad \Rightarrow \quad x_1^k \equiv x_2^k \mod n.$$

♠ *Preuve* – Si $x_1 \equiv x_2 \mod n$ et $y_1 \equiv y_2 \mod n$, alors il existe $k, \ell \in \mathbb{Z}$ tels que $x_1 = x_2 + kn$ et $y_1 = y_2 + \ell n$. Donc

$$\begin{aligned} x_1 + y_1 &= (x_2 + kn) + (y_2 + \ell n) \\ &= (x_2 + y_2) + (k + \ell)n, \\ x_1 \times y_1 &= (x_2 + kn)(y_2 + \ell n) \\ &= x_2 y_2 + (x_2 \ell + y_2 k + k\ell n)n. \end{aligned}$$

Donc $x_1 + y_1 \equiv x_2 + y_2 \mod n$ et $x_1 \times y_1 \equiv x_2 \times y_2 \mod n$. ■

**Le classe de congruence** de $x$ modulo $n$, notée $[x]_n$, est l'ensemble

$$[x]_n := \{y \in \mathbb{Z} \mid y \equiv x \mod n\} = \{y \in \mathbb{Z} \mid y\%n = x\%n\}.$$

Étant donnée une classe de congruence $a$, on peut en choisir un *représentant* $x \in a$ tel que $a = [x]_n = \{x + k \times n,\ k \in \mathbb{Z}\}$. Ce représentant n'est pas unique : $a = [x]_n = [x + n]_n = [x + 2n]_n = \ldots$. Par exemple, 2, 7 et $-3$ sont trois représentants de la classe $[2]_5$, puisque

$$[2]_5 = \{\ldots, -8, -3, 2, 7, 12, \ldots\} = \{2 + k \times 5,\ k \in \mathbb{Z}\}.$$

**Exercice 2.2**

Citer cinq éléments de $[-2]_7$, la classe de congruence de $-2$ modulo 7.

**Structure algébrique.** On peut étendre les opérations aux classes de congruence.

- *Addition :* $[x]_n + [y]_n := [x+y]_n$
- *Soustraction :* $[x]_n - [y]_n := [x-y]_n$
- *Multiplication :* $[x]_n \times [y]_n := [x \times y]_n$

Ces nouvelles opérations sont bien définies puisque les classes de congruence à droite des égalités sont indépendantes du choix des représentants $x$ et $y$ d'après la Proposition 2.3.

*Exemple* 2.4. Modulo 5, on a $[3]_5 \times [4]_5 = [3 \times 4]_5 = [12]_5 = [2]_5$. Une autre manière de le calculer est d'utiliser $[3]_5 = [-2]_5$ et $[4]_5 = [-1]_5$, d'où

$$[3]_5 \times [4]_5 = [-2]_5 \times [-1]_5 = [(-2) \times (-1)]_5 = [2]_5.$$

**Exercice 2.3**

Calculer les restes modulo 10 de $122+455$, $122\times455$, $122^{455}$. Mêmes calculs modulo 11, puis modulo 12.

**Anneau résiduel.** Cela permet de définir un nouvel anneau dont les éléments sont les classes de congruence.

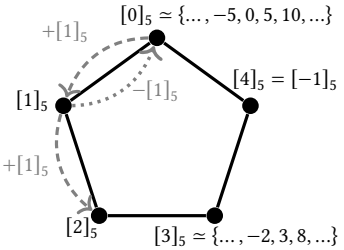



**Figure 2.2. :** Représentation de l'anneau résiduel $\mathbb{Z}_5$.

**Anneau résiduel $\mathbb{Z}_n$**

Pour $n$ un entier naturel non nul, on définit l'ensemble des nombres résiduels (ou modulaires) par

$$\mathbb{Z}_n := \{[x]_n,\ x \in \mathbb{Z}\}.$$

Cet ensemble, appelé *anneau résiduel* ou *anneau des entiers modulo n*, est souvent noté $\mathbb{Z}/n\mathbb{Z}$ dans la littérature.

Si les entiers relatifs sont représentés en ligne, on représente généralement les entiers modulaires sur un cercle, comme en Figure 2.2. Ce nouvel anneau est fini : il contient exactement $n$ éléments, qui sont les classes de congruence

$$\mathbb{Z}_n = \{[0]_n, [1]_n, [2]_n, ..., [n-1]_n\}.$$

Pour « annuler » [#4] la conversion entier → résiduel du crochet $[\cdot]_n$, on peut choisir un *représentant canonique* de la classe de congruence. On définit pour cela une fonction $\iota : \mathbb{Z}_n \to \mathbb{Z}$ définie par

$$\iota([x]_n) := x\%n \in \{0, 1, ..., n-1\}.$$

Le résultat est indépendant du choix du représentant $x$ de la classe $[x]_n$, donc la fonction est bien définie.

#4 : Étant donnée une classe $a = [x]_n$, on a $[\iota(a)]_n = a$, mais pas forcément $\iota([x]_n) = x$. Par exemple, $\iota([7]_5) = 2$. Ainsi, l'injection $\iota$ n'annule pas la conversion en général, elle envoie juste sur le représentant canonique.

**Le calcul sur les représentants n'est pas le même que le calcul sur les classes**

Alors qu'on a toujours $[x+y]_n = [x]_n + [y]_n$, on a parfois

$$(x+y)\%n \neq (x\%n) + (y\%n).$$

C'est vrai même si on utilise le représentant canonique dans les calculs ! Par exemple, $(3+3)\%5 = 6\%5 = 1$ alors que $(3\%5) + (3\%5) = 3+3 = 6 \neq 1$. Lorsqu'on effectue des calculs, il faut donc appliquer le modulo régulièrement.

**Exercice 2.4** **Divisibilité par 3**

Un entier $x$ est divisible par un naturel $n$ si $x\%n = 0$. On dénote $(c_j)_j$ les chiffres d'un entier naturel $x$ en écriture décimale, de sorte que $x = c_0 + 10c_1 + ... + 10^r c_r$ avec $c_j < 10$ pour tout $j$. Montrer que $x$ est divisible par 3 si et seulement si la somme de ses chiffres est divisible par 3.

Démontrer la même propriété pour la divisibilité par 9. Est-ce qu'il existe d'autres nombres entre 1 et 9 qui ont cette propriété ?

### 2.1.4. ♠ Une autre définition de l'anneau résiduel

**Proposition 2.5.** *Pour $n \in \mathbb{N}^*$ fixé, la relation de congruence modulo $n$ est une relation d'équivalence sur $\mathbb{Z}$, c'est-à-dire qu'elle est*

1. réflexive *: $x \equiv x \mod n$*
2. symétrique *: $x \equiv y \mod n \quad \Leftrightarrow \quad y \equiv x \mod n$*
3. transitive *: si $x \equiv y \mod n$ et $y \equiv z \mod n$*

De ce fait, les classes de congruence sont des classes d'équivalence. Elles forment donc une partition de $\mathbb{Z}$ : chaque entier relatif appartient à une et une seule classe de congruence modulo $n$.

**Exercice 2.5** ♠

*Prouver* que la relation de congruence modulo $n$ est une relation d'équivalence.

*Remarque* 2.6. De manière générale, si on a une fonction $f$, on peut considérer que deux éléments $x, y$ sont en relation si $f(x) = f(y)$. Autrement dit, une relation d'équivalence peut être de la forme « $x$ a la même propriété que $y$ » (e.g. deux livres ont le même ISBN, deux billes ont la même couleur...). Ici la fonction est $x \mapsto x\%n$, le reste de la division euclidienne par $n$. //

Avec cette définition, les opérations algébriques ($+$, $-$ ou $\times$) n'est pas intrinsèque. Dans le diagramme ci-dessous, pour définir le résultat d'un opérateur $\star$ en pointillés, il faut suivre le long chemin en flèches pleines, où $\iota \otimes \iota$ est la fonction $(a,b) \mapsto \big(\iota(a), \iota(b)\big)$.

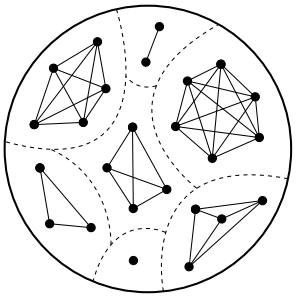
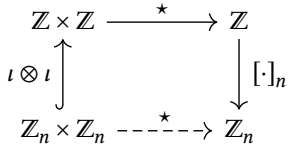


**Figure 2.3. :** Graphe d'un exemple de relation d'équivalence avec 7 classes. Les classes sont distinguées par les zones en pointillées (source *Wikimédia Commons*).




À partir de deux modulaires $a, b \in \mathbb{Z}_n$, on choisit des représentants $x, y \in \mathbb{Z}$ tels que $a = [x]_n$ et $b = [y]_n$, puis on pose $a \star b = [x \star y]_n$.

## 2.2. Exercices du chapitre

**Exercice 2.1**

Sachant que l'on a $96\,842 = 256 \times 375 + 842$, déterminer, sans faire la division, le reste de la division du nombre 96 842 par chacun des nombres 256 et 375.

**Exercice 2.2**

Citer cinq éléments de $[-2]_7$, la classe de congruence de $-2$ modulo 7.

**Exercice 2.3**

Calculer les restes modulo 10 de $122 + 455$, $122 \times 455$, $122^{455}$. Mêmes calculs modulo 11, puis modulo 12.

**Exercice 2.4** **Divisibilité par 3**

Un entier $x$ est divisible par un naturel $n$ si $x\%n = 0$. On dénote $(c_j)_j$ les chiffres d'un entier naturel $x$ en écriture décimale, de sorte que $x = c_0 + 10c_1 + ... + 10^r c_r$ avec $c_j < 10$ pour tout $j$. Montrer que $x$ est divisible par 3 si et seulement si la somme de ses chiffres est divisible par 3.

Démontrer la même propriété pour la divisibilité par 9. Est-ce qu'il existe d'autres nombres entre 1 et 9 qui ont cette propriété ?

**Exercice 2.5** ♠

*Prouver* que la relation de congruence modulo $n$ est une relation d'équivalence.

# Appendices

# Informations vis-à-vis du cours | A.

## A.1. Programme du cours

### A.1.1. Programme national

**Objectif** L'objectif de cette ressource est d'introduire les diverses techniques employées en cryptographie.

**Savoirs de référence étudiés**

- Arithmétique ($\mathbb{Z}/n\mathbb{Z}$, groupe cyclique...)
- Introduction à la cryptographie symétrique (par ex. : César, Vigenère, Hill...)
- Cryptographie asymétrique (par ex. : chiffrement RSA, Diffie-Hellman, El Gamal...)
- Initiation aux codes détecteurs et correcteurs (par ex. : Hamming, bits de parité...)

**Prolongements suggérés**

- Hachage, signature, intégrité
- Stockage des mots de passe

**Mots-clés**

Cryptographie, Arithmétique, Chiffrement, Contrôle de parité

**Répartition des heures**

- TD : 10.5h (7 séances)
- TP : 15h (10 séances)
- Travail personnel : 10 minutes entre chaque séance (90% du travail en cours, 10% en autonomie)

**Compétences visées**

- Compétence 2, AC3 : Comprendre la sécurisation
- Compétence 3, AC3 : Sécuriser un système
- Compétence 4, AC2 : Assurer la confidentialité des données

Compétence 2 : Sélectionner les algorithmes adéquats pour répondre à un problème donné

Compétence 3 : Déployer des services dans une architecture réseau

Compétence 4 : Optimiser une base de données, interagir avec une application et mettre en œuvre la sécurité

### A.1.2. Contrat pédagogique

La ressource « R3.09 - Cryptographie » a pour objectif de fournir aux étudiants les connaissances et compétences nécessaires pour comprendre les principes fondamentaux de la cryptographie, ainsi que leur application dans le domaine de la sécurité informatique.

**Qu'est-ce que cette ressource va m'apporter ?**

Le programme national étant plutôt flou, précisons les compétences visées par cette ressource. Dans ce cours à l'IUT de Lens, elles sont regroupées en trois catégories : la culture cryptographique, les mathématiques et la programmation.

*Culture cryptographique :*

- Connaitre les algorithmes de chiffrement et de hachage courants et leur contexte d'utilisation
- Décrire les éléments d'un canal de communication sécurisé

*Mathématiques :*

- Concepts mathématiques : structures algébriques, arithmétique modulaire, théorie des nombres
- Calculer un échange de clés Diffie-Hellman
- Prouver le bon fonctionnement de la signature RSA

*Programmation :*

- OpenSSL : générer et manipuler de clés, chiffrer/déchiffrer, signer/valider
- Manipuler de grands entiers en Python

**À quoi cela va me servir ?**

- Sélectionner un algorithme de chiffrement ou de hachage adapté à un contexte donné.
- Alerter sur des failles de sécurité basiques dans un système informatique.
- Découvrir et manipuler des outils mathématiques de théorie des nombres.

**Comment vont se dérouler les évaluations ?**

- Évaluation papier : questions de cours, QCM, exercices de calculs.
- Évaluation pratique : TP sur ordinateur, manipulation d'outils de cryptographie.
- Auto-évaluation régulière : QCM en début de séance, exercices supplémentaires facultatifs.

**Comment vont se dérouler les séances ?** Les TD se concentrent sur les mathématiques et la théorie de la cryptographie.

Les TP cherchent à illustrer les concepts mathématiques vus en TD, mais aussi à manipuler des outils de cryptographie plus concret (comme OpenSSL).

**À quoi je m'engage ?**

- À participer activement aux activités proposées dans les séances.
- À avoir un usage responsable des modèles génératifs de langage (IA type ChatGPT, Claude, etc).
- À poursuivre le travail en dehors des séances si le besoin s'en fait ressentir.
- À justifier de vos absences et retards éventuels, en particulier lors des évaluations.

**À quoi s'engage l'enseignant ?**

- À rester à l'écoute de vos remarques, et à répondre à vos questions de manière claire et précise.
- À mettre à disposition un support de cours (poly mis à jour régulièrement) et exercices corrigés (TP et TD) sur *Moodle*.
- À se rendre disponible par mail à l'adresse `leopold.tremant@univ-artois.fr` ou dans le bureau 005E.

## A.2. Ressources complémentaires

Une partie de ce cours est inspirée du livre de Rémi Boulle, *La cryptographie par la pratique avec Python et OpenSSL*.

Le livre en anglais *Crypto 101*, disponible gratuitement en ligne, est également une excellente ressource qui s'intéresse à la pratique autant qu'à la théorie.

Pour mettre les mains dans le cambouis, le site CryptoPals propose des défis de cryptographie, qui sont très intéressants pour approfondir les aspects pratico-pratiques de la cryptographie.

Sur YouTube, on trouve aussi quelques créateur-ices qui parlent de cryptographie :

- Science4All (en français)
- Computerphile (en anglais)
- Purple Mind (en anglais)

# Solutions aux exercices

## Chapitre 1

**Solution à l'Exercice 1.1** ▲ page 8

1. Avec dCode, le décryptage automatique fonctionne immédiatement pour retrouver « Bonjour » à partir de `Erqmrxu`.

| ↑↓ | ↑↓ |
|---|---|
| →3 (←23) | Bonjour |
| →12 (←14) | Sfeafli |
| →4 (←22) | Anmintq |
| →9 (←17) | Vihdiol |
| →23 (←3) | Hutpuax |

2. a) En chiffrant « Zut » avec un décalage de 10, on obtient le message `Jkd`.

| | |
|---|---|
| ←10 (→16) | Jed |
| →10 (←16) | Pkj |

   b) Le décryptage automatique du message `Jkd` suggère principalement un autre message en clair : « Ton ». Bien que ce soit un mot français, ce n'est pas le message original. La suggestion « Zut » est proposée en 9ème position.

| ↑↓ | ↑↓ |
|---|---|
| →16 (←10) | Ton |
| →15 (←11) | Upo |
| →9 (←17) | Avu |
| →1 (←25) | Idc |
| →5 (←21) | Ezy |
| →3 (←23) | Gba |
| →11 (←15) | Yts |
| →22 (←4) | Nih |
| →10 (←16) | Zut |
| →4 (←22) | Faz |

   c) Malgré son manque de sûreté sur les longs messages, le chiffre de César peut être relativement sécurisé sur de très courts messages (comme ici avec « Zut »). Il est possible que le décryptage automatique ne retrouve pas le message original, car trop de mots différents sont probables. C'est le principe du masque jetable : changer le décalage à chaque lettre rend le décryptage automatique impossible.

**Solution à l'Exercice 1.2** ▲ page 9

On va noter dénoter avec la couleur verte les secrets d'Alice, et avec la couleur orange ceux de Bob.

Calculons d'abord le secret partagé

$$Z = G^{a \times b} \% p = 5^{2 \times 3} \% 23 = 5^6 \% 23 = 15\,625 \% 23 = 8.$$

On espère donc trouver $Z_a = Z_b = 8$. De son côté, Alice commence par calculer son message public

$$A = G^{a} \% p = 5^{2} \% 23 = 25 \% 23 = 2,$$

et Bob calcule le sien

$$B = G^{b} \% p = 5^{3} \% 23 = 125 \% 23 = 10.$$

Une fois les messages publics échangés, Alice connait $B$ et Bob connait $A$. Ils calculent le secret partagé

$$Z_a = B^a \% p = 10^2 \% 23 = 100 \% 23 = 8,$$
$$Z_b = A^b \% p = 2^3 \% 23 = 8 \% 23 = 8.$$

On obtient bien le même secret partagé $Z_a = Z_b = Z = 8$.

# Chapitre 2

**Solution à l'Exercice 2.1** ▲ page 16

- Pour la division par 256, on a $842 = 3 \times 256 + 74$, donc le reste est 74.
- Pour la division par 375, on a $842 = 2 \times 375 + 92$, donc le reste est 92.

**Solution à l'Exercice 2.2** ▲ page 16

La classe de congruence de $-2$ modulo 7 est

$$[-2]_7 = [5]_7 = \{..., -9, -2, 5, 12, 19, ...\} = \{2 + 7 \times k,\ k \in \mathbb{Z}\}$$

**Solution à l'Exercice 2.3** ▲ page 16

*Modulo 10.* On a $122 \equiv 2 \mod 10$ et $455 \equiv 5 \mod 10$, donc

- $122 + 455 \equiv 2 + 5 \equiv 7 \mod 10$,
- $122 \times 455 \equiv 2 \times 5 \equiv 0 \mod 10$,
- $122^{455} \equiv 2^{455} \mod 10$. On calcule ensuite les puissances successives

$$2^2 = 4 \equiv 4 \mod 10,$$
$$2^3 = 8 \equiv 8 \mod 10,$$
$$2^4 = 16 \equiv 6 \mod 10,$$
$$2^5 = 2^4 \times 2 \equiv 6 \times 2 = 12 \equiv 2 \mod 10.$$

On reconnait ici $2^5 \equiv 2^1 \mod 10$. On en déduit que la suite des restes modulo 10 est périodique de période 4, de sorte que pour toute puissance $k > 1$, on a

$$122^k \equiv 2^k \equiv \begin{cases} 6 & \text{si } k \equiv 0 \mod 4, \\ 2 \mod 10 & \text{si } k \equiv 1 \mod 4, \\ 4 \mod 10 & \text{si } k \equiv 2 \mod 4, \\ 8 \mod 10 & \text{si } k \equiv 3 \mod 4. \end{cases}$$

Grâce à la division euclidienne $455 = 113 \times 4 + 3$, on trouve finalement $2^{455} \equiv 2^3 \equiv 8 \mod 10$.

*Modulo 11.* On a $122 \equiv 1 \mod 11$ et $455 \equiv 3 \mod 11$, donc

- $122 + 455 \equiv 1 + 3 \equiv 4 \mod 11$,
- $122 \times 455 \equiv 1 \times 3 \equiv 3 \mod 11$,
- $122^{455} \equiv 1^{455} \equiv 1 \mod 11$.

*Modulo 12.* On a $122 \equiv 2 \mod 12$ et $455 \equiv 11 \mod 12$, donc

- $122 + 455 \equiv 2 + 11 \equiv 1 \mod 12$,
- $122 \times 455 \equiv 2 \times 11 \equiv 10 \mod 12$,
- $122^{455} \equiv 2^{455} \mod 12$. On calcule ensuite les puissances successives

$$\begin{aligned} 2^2 &= 4 \equiv 4 \mod 12, \\ 2^3 &= 8 \equiv 8 \mod 12, \\ 2^4 &= 16 \equiv 4 \mod 12, \end{aligned}$$

On reconnait ici $2^4 \equiv 2^2 \mod 12$. On en déduit que la suite des restes modulo 12 est périodique de période 2, de sorte que pour toute puissance $k > 1$, on a

$$122^k \equiv 2^k \equiv \begin{cases} 4 \mod 12 & \text{si } k \equiv 0 \mod 2, \\ 8 \mod 12 & \text{si } k \equiv 1 \mod 2. \end{cases}$$

Grâce à la division euclidienne $455 = 227 \times 2 + 1$, on trouve finalement $2^{455} \equiv 8 \mod 12$.

**Solution à l'Exercice 2.4** ▲ page 16

Par définition, $x$ est divisible par 3 si $x\%n = 0$, i.e. si $x \equiv 0 \mod 3$. Or on a $10 \equiv 1 \mod 3$, d'où $10^k \equiv 1^k \equiv 1 \mod 3$ pour tout $k$, de sorte que

$$x = a_0 + 10a_1 + ... + 10^r a_r \equiv a_0 + a_1 + ... + a_r \mod 3.$$

Ainsi,

$$x \equiv 0 \mod 3 \quad \Leftrightarrow \quad a_0 + a_1 + ... + a_r \equiv 0 \mod 3,$$

autrement dit $x$ est divisible par 3 si et seulement si la somme de ses chiffres est divisible par 3.

Le même raisonnement s'applique pour la divisibilité par 9, puisque $10 \equiv 1 \mod 9$. En revanche, aucun autre nombres entre 1 et 9 ne satisfait la même propriété : $10 \equiv 0 \not\equiv 1 \mod 1$, puis

$$\begin{aligned} &10 \equiv 0 \not\equiv 1 \mod 2, \quad 10 \equiv 2 \not\equiv 1 \mod 4, \quad 10 \equiv 0 \not\equiv 1 \mod 5, \\ &10 \equiv 4 \not\equiv 1 \mod 6, \quad 10 \equiv 3 \not\equiv 1 \mod 7, \quad 10 \equiv 2 \not\equiv 1 \mod 8. \end{aligned}$$

**Solution à l'Exercice 2.5** ▲ page 16

- Réflexivité : $a \equiv a \pmod n$ car $a - a = 0$ est divisible par $n$.
- Symétrie : si $a \equiv b \pmod n$, alors $b \equiv a \pmod n$ car si $a - b$ est divisible par $n$, alors $b - a = -(a - b)$ est aussi divisible par $n$.
- Transitivité : si $a \equiv b \pmod n$ et $b \equiv c \pmod n$, alors $a \equiv c \pmod n$ car $(a - b) + (b - c) = a - c$ est la somme de deux entiers divisibles par $n$, donc elle est aussi divisible par $n$.

Donc la relation de congruence modulo $n$ est réflexive, symétrique et transitive, ce qui en fait une relation d'équivalence.